

```
# This will show an UPLOAD BUTTON – click it and navigate to:
# C:\Users\sreya\Desktop\MINOR\ASS_S6_2\archive.zip
# and select that file
```

```
from google.colab import files
print("👉 Click the button below and upload archive.zip from your Desktop")
uploaded = files.upload()
print("✅ Upload complete!")
```

Enable browser notifications
in Settings to get alerts when
executions complete

OK No thanks

👉 Click the button below and upload archive.zip from your Desktop

Choose Files archive.zip

archive.zip(application/x-zip-compressed) - 164670110 bytes, last modified: 10/3/2026 - 100% done

Saving archive.zip to archive.zip

✅ Upload complete!

```
import os
```

```
# Extract the zip file
!unzip -q archive.zip -d /content/brain_tumor
```

```
# Show what's inside
print("📁 Contents of extracted folder:")
for item in os.listdir('/content/brain_tumor'):
    print("📁", item)
```

📁 Contents of extracted folder:

📁 Training

📁 Testing

```
import os
```

```
# Set paths – run Cell 2 first and check the folder names printed above
# Most likely it will be 'Training' and 'Testing'
train_dir = '/content/brain_tumor/Training'
test_dir = '/content/brain_tumor/Testing'
```

```
# Verify folders exist
if os.path.exists(train_dir):
    print("✅ Training folder found!")
else:
    print("❌ Training folder NOT found – check folder name above and update train_dir")
```

```
if os.path.exists(test_dir):
    print("✅ Testing folder found!")
else:
    print("❌ Testing folder NOT found – check folder name above and update test_dir")
```

```
# Show classes and image counts
classes = sorted(os.listdir(train_dir))
print("\n📁 Tumor Classes Found:", classes)
print()
print(f"{'Class':<15} {'Train Images':>15} {'Test Images':>12}")
print("-" * 45)
for cls in classes:
    train_count = len(os.listdir(os.path.join(train_dir, cls)))
    test_count = len(os.listdir(os.path.join(test_dir, cls)))
    print(f"{cls:<15} {train_count:>15} {test_count:>12}")
```

✅ Training folder found!

✅ Testing folder found!

📁 Tumor Classes Found: ['glioma', 'meningioma', 'notumor', 'pituitary']

Class	Train Images	Test Images
glioma	1400	400
meningioma	1400	400
notumor	1400	400
pituitary	1400	400

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```

from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing import image
from sklearn.metrics import classification_report, confusion_matrix

```

```

print("✅ All libraries imported!")
print("✅ TensorFlow version:", tf.__version__)

```

```

✅ All libraries imported!
✅ TensorFlow version: 2.19.0

```

Enable browser notifications
in Settings to get alerts when
executions complete

```

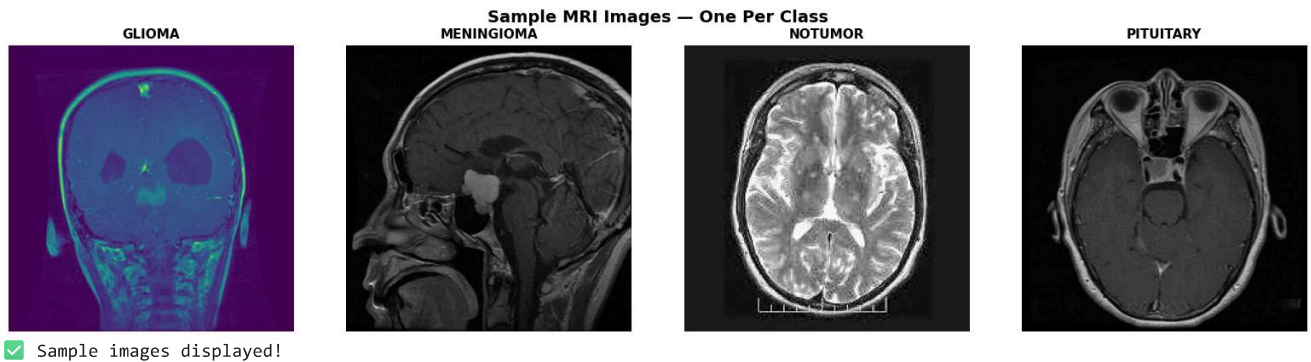
fig, axes = plt.subplots(1, len(classes), figsize=(16, 4))
fig.suptitle("Sample MRI Images – One Per Class", fontsize=14, fontweight='bold')

for i, cls in enumerate(classes):
    cls_folder = os.path.join(train_dir, cls)
    img_name = os.listdir(cls_folder)[0]
    img_path = os.path.join(cls_folder, img_name)

    img = plt.imread(img_path)
    axes[i].imshow(img)
    axes[i].set_title(cls.upper(), fontsize=11, fontweight='bold')
    axes[i].axis('off')

plt.tight_layout()
plt.show()
print("✅ Sample images displayed!")

```



```

IMG_SIZE = 150 # resize all images to 150x150 pixels
BATCH_SIZE = 32 # process 32 images at a time

# Training data – with augmentation (creates variations to improve learning)
train_datagen = ImageDataGenerator(
    rescale=1./255, # normalize pixel values from 0-255 to 0-1
    rotation_range=10, # randomly rotate images slightly
    horizontal_flip=True, # randomly flip images left-right
    zoom_range=0.1, # randomly zoom in slightly
    validation_split=0.2 # use 20% of training images for validation
)

# Test data – only normalize, no augmentation
test_datagen = ImageDataGenerator(rescale=1./255)

# Load training images (80% of Training folder)
train_data = train_datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='training'
)

# Load validation images (20% of Training folder)
val_data = train_datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='validation'
)

```

```

)

# Load test images (from Testing folder)
test_data = test_datagen.flow_from_directory(
    test_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False # keep order for evaluation
)

print("\n✅ Data loaded successfully!")
print("📁 Class mapping:", train_data.class_indices)
print(f"📁 Training batches : {len(train_data)}")
print(f"📁 Validation batches: {len(val_data)}")
print(f"📁 Test batches      : {len(test_data)}")

```

Found 4480 images belonging to 4 classes.
 Found 1120 images belonging to 4 classes.
 Found 1600 images belonging to 4 classes.

```

✅ Data loaded successfully!
📁 Class mapping: {'glioma': 0, 'meningioma': 1, 'notumor': 2, 'pituitary': 3}
📁 Training batches : 140
📁 Validation batches: 35
📁 Test batches      : 50

```

Enable browser notifications
 in Settings to get alerts when
 executions complete

```

NUM_CLASSES = len(classes) # automatically detects number of classes (4)

model = models.Sequential([

    # Block 1 – detect basic features like edges
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(150, 150, 3)),
    layers.MaxPooling2D(2, 2),

    # Block 2 – detect shapes and patterns
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(2, 2),

    # Block 3 – detect complex tumor features
    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D(2, 2),

    # Flatten and classify
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5), # reduces overfitting
    layers.Dense(NUM_CLASSES, activation='softmax') # final output layer

], name="Brain_Tumor_CNN")

# Compile the model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
print("\n✅ CNN Model built successfully!")

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape` to `Model.add\_layer`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "Brain Tumor CNN"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
flatten (Flatten)	(None, 36992)	0
dense (Dense)	(None, 128)	4,735,104
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 4)	516

Total params: 4,828,868 (18.42 MB)
Trainable params: 4,828,868 (18.42 MB)
Non-trainable params: 0 (0.00 B)

✅ CNN Model built successfully!

Enable browser notifications
in Settings to get alerts when
executions complete

```
print("Training started...")  
print("This will take around 5-10 minutes with GPU enabled\n")
```

```
history = model.fit(  
    train_data,  
    epochs=15,  
    validation_data=val_data,  
    verbose=1  
)
```

```
print("\n✅ Training complete!")
```

Training started...
This will take around 5-10 minutes with GPU enabled

```
Epoch 1/15  
140/140 ————— 263s 2s/step - accuracy: 0.5949 - loss: 0.9191 - val_accuracy: 0.7518 - val_loss: 0.6191  
Epoch 2/15  
140/140 ————— 253s 2s/step - accuracy: 0.7489 - loss: 0.6318 - val_accuracy: 0.7884 - val_loss: 0.5088  
Epoch 3/15  
140/140 ————— 262s 2s/step - accuracy: 0.7926 - loss: 0.5415 - val_accuracy: 0.8054 - val_loss: 0.4676  
Epoch 4/15  
140/140 ————— 261s 2s/step - accuracy: 0.8076 - loss: 0.4860 - val_accuracy: 0.8277 - val_loss: 0.4605  
Epoch 5/15  
140/140 ————— 251s 2s/step - accuracy: 0.8330 - loss: 0.4336 - val_accuracy: 0.8366 - val_loss: 0.3939  
Epoch 6/15  
140/140 ————— 261s 2s/step - accuracy: 0.8460 - loss: 0.3871 - val_accuracy: 0.8250 - val_loss: 0.4384  
Epoch 7/15  
140/140 ————— 251s 2s/step - accuracy: 0.8538 - loss: 0.3752 - val_accuracy: 0.8339 - val_loss: 0.4019  
Epoch 8/15  
140/140 ————— 260s 2s/step - accuracy: 0.8690 - loss: 0.3296 - val_accuracy: 0.8705 - val_loss: 0.3489  
Epoch 9/15  
140/140 ————— 253s 2s/step - accuracy: 0.8824 - loss: 0.3153 - val_accuracy: 0.8839 - val_loss: 0.3229  
Epoch 10/15  
140/140 ————— 251s 2s/step - accuracy: 0.9002 - loss: 0.2618 - val_accuracy: 0.8893 - val_loss: 0.3060  
Epoch 11/15  
140/140 ————— 261s 2s/step - accuracy: 0.9092 - loss: 0.2472 - val_accuracy: 0.8946 - val_loss: 0.2778  
Epoch 12/15  
140/140 ————— 270s 2s/step - accuracy: 0.9089 - loss: 0.2418 - val_accuracy: 0.9027 - val_loss: 0.2662  
Epoch 13/15  
140/140 ————— 261s 2s/step - accuracy: 0.9170 - loss: 0.2250 - val_accuracy: 0.9179 - val_loss: 0.2402  
Epoch 14/15  
140/140 ————— 249s 2s/step - accuracy: 0.9185 - loss: 0.2074 - val_accuracy: 0.9259 - val_loss: 0.2156  
Epoch 15/15  
140/140 ————— 269s 2s/step - accuracy: 0.9228 - loss: 0.2042 - val_accuracy: 0.9071 - val_loss: 0.2338
```

✅ Training complete!

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))  
fig.suptitle("CNN Model Training Results", fontsize=14, fontweight='bold')
```

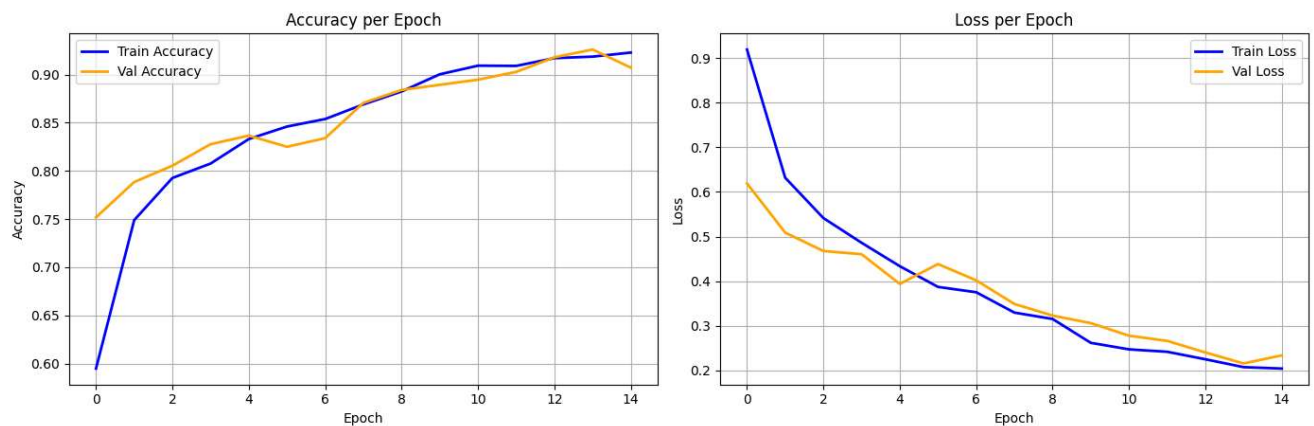
```
# Accuracy graph
axes[0].plot(history.history['accuracy'], label='Train Accuracy', color='blue', linewidth=2)
axes[0].plot(history.history['val_accuracy'], label='Val Accuracy', color='orange', linewidth=2)
axes[0].set_title('Accuracy per Epoch')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True)

# Loss graph
axes[1].plot(history.history['loss'], label='Train Loss', color='blue', linewidth=2)
axes[1].plot(history.history['val_loss'], label='Val Loss', color='orange', linewidth=2)
axes[1].set_title('Loss per Epoch')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
print("✅ Training graphs shown!")
```

Enable browser notifications
in Settings to get alerts when
executions complete

CNN Model Training Results



✅ Training graphs shown!

```
print("🔍 Evaluating on test images...\n")

test_loss, test_accuracy = model.evaluate(test_data, verbose=0)

print(f"📊 Test Accuracy : {test_accuracy * 100:.2f}%")
print(f"📊 Test Loss : {test_loss:.4f}")
```

🔍 Evaluating on test images...

📊 Test Accuracy : 82.69%
📊 Test Loss : 0.9129

```
# Get predictions
test_data.reset()
predictions = model.predict(test_data, verbose=0)
predicted_classes = np.argmax(predictions, axis=1)
true_classes = test_data.classes
class_names = list(test_data.class_indices.keys())

# Classification report
print("📄 Classification Report:")
print("=" * 60)
print(classification_report(true_classes, predicted_classes, target_names=class_names))

# Confusion matrix
cm = confusion_matrix(true_classes, predicted_classes)
plt.figure(figsize=(8, 6))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cbar=True)
```

```

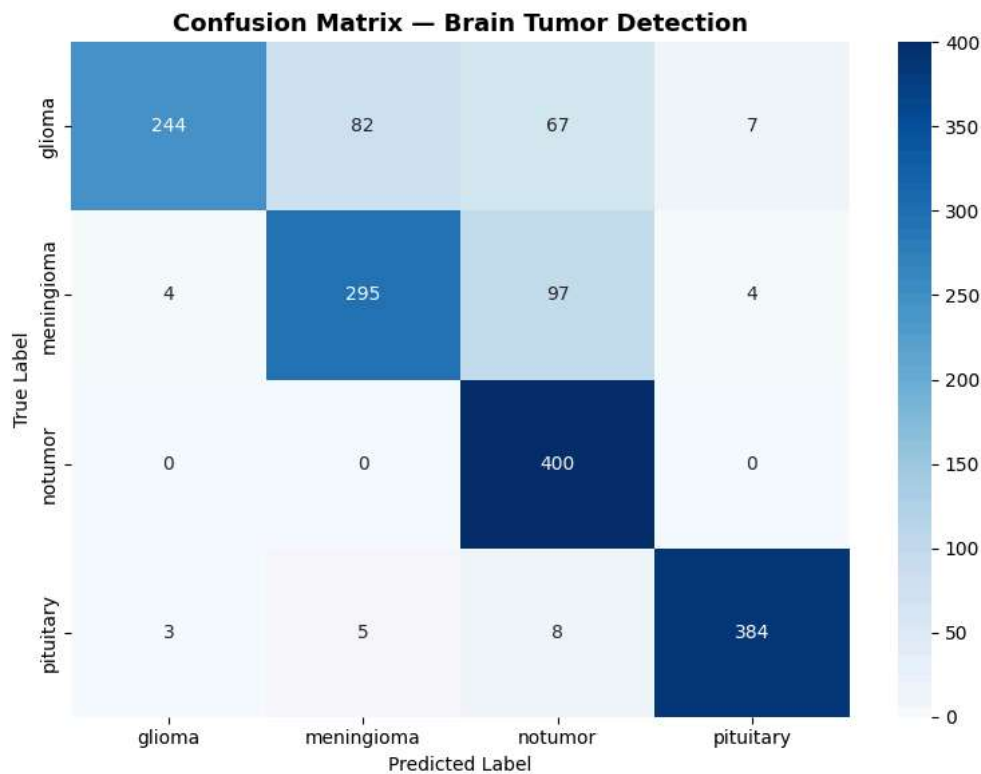
cmap= Blues ,
xticklabels=class_names,
yticklabels=class_names)
plt.title('Confusion Matrix - Brain Tumor Detection', fontsize=13, fontweight='bold')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.show()
print("✅ Evaluation complete!")

```

Enable browser notifications
in Settings to get alerts when
executions complete

📄 Classification Report:

	precision	recall	f1-score	support
glioma	0.97	0.61	0.75	400
meningioma	0.77	0.74	0.75	400
notumor	0.70	1.00	0.82	400
pituitary	0.97	0.96	0.97	400
accuracy			0.83	1600
macro avg	0.85	0.83	0.82	1600
weighted avg	0.85	0.83	0.82	1600



✅ Evaluation complete!

```

# Automatically picks one test image to predict
sample_class = classes[0]
sample_folder = os.path.join(test_dir, sample_class)
img_path = os.path.join(sample_folder, os.listdir(sample_folder)[0])

# Prepare image
img = image.load_img(img_path, target_size=(150, 150))
img_array = image.img_to_array(img) / 255.0
img_array = np.expand_dims(img_array, axis=0)

# Predict
pred = model.predict(img_array, verbose=0)[0]
pred_class = class_names[np.argmax(pred)]
confidence = np.max(pred) * 100

# Show result
plt.figure(figsize=(5, 5))
plt.imshow(image.load_img(img_path))
plt.axis('off')
color = 'red' if pred_class != 'notumor' else 'green'
plt.title(f"Predicted: {pred_class.upper()}\nConfidence: {confidence:.1f}%",
         fontsize=13, fontweight='bold', color=color)

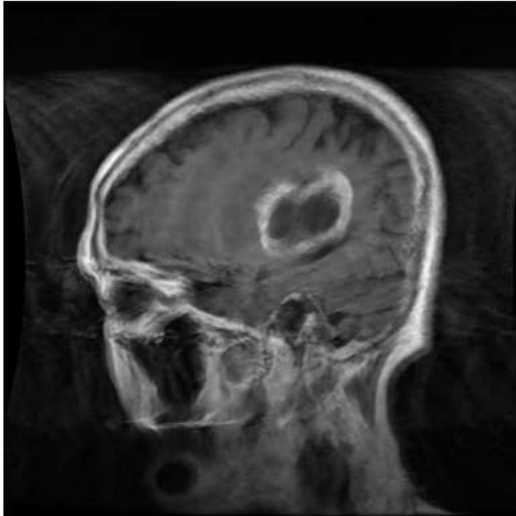
```

```
plt.show()
```

```
print("\n📊 All Class Probabilities:")
for cls, prob in zip(class_names, pred):
    bar = '█' * int(prob * 40)
    print(f" {cls:15s}: {prob*100:5.1f}% {bar}")
```

Enable browser notifications
in Settings to get alerts when
executions complete

Predicted: MENINGIOMA
Confidence: 57.7%



```
📊 All Class Probabilities:
```

```
glioma       : 41.8% ██████████
meningioma   : 57.7% ██████████
notumor      : 0.1% 
pituitary    : 0.4%
```

```
# Save model
model.save('brain_tumor_model.h5')
print("✅ Model saved!")
```

```
# Download to your computer
from google.colab import files
files.download('brain_tumor_model.h5')
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format
✅ Model saved!